Explication du mécanisme des comptes de service SQL Server

Conclusion préalable

Nous venons de mettre en place le socle de sécurité sur lequel SQL Server va s'appuyer pendant toute sa durée de vie. Les deux comptes `SVC_SQL_Engine` et `SVC_SQL_Agent` ne sont pas de simples comptes AD : ce sont les **identités sous lesquelles les services SQL Server vont tourner en permanence**. Les deux privilèges qu'on vient de leur accorder (‘*Log on as a service’* et ‘*Perform volume maintenance tasks*’) sont **indispensables** au bon fonctionnement de SQL Server, et **leur absence bloquerait l'installation ou dégraderait les performances**.

Voici l'explication complète du mécanisme, du pourquoi et du comment.

---

1. Les deux comptes de service

1.1 `SVC_SQL_Engine` — le moteur SQL Server

Rôle : c'est le compte sous lequel tourne le **service principal** `MSSQLSERVER`. C'est **l'identité de SQL Server lui-même**.

Ce qu'il fait :

- Démarre le service `MSSQLSERVER` au boot de la machine.
- Se connecte aux fichiers de données (`S:\SQL\Data`), de logs (`S:\SQL\Log`), de sauvegardes (`S:\SQL\Backup`) et TempDB (`S:\SQL\TempDB`).
- Écrit dans les journaux d'erreurs SQL Server.
- Se connecte aux autres nœuds du cluster Always On (endpoint 5022).
- Lit et écrit dans la base `master`, `msdb`, `model`, `tempdb`.

Sans ce compte, SQL Server ne peut **pas démarrer**.

1.2 `SVC_SQL_Agent` — l'agent SQL Server

Rôle : c'est le compte sous lequel tourne le **service `SQLSERVERAGENT`**. C'est **l'identité du planificateur de tâches SQL Server**.

Ce qu'il fait :

- Exécute les **jobs SQL Server Agent** (backups, DBCC CHECKDB, index rebuild).
- Se connecte à SQL Server via **Windows Authentication**.
- Écrit dans la base `msdb` (qui contient la définition des jobs).
- Déclenche les alertes configurées (severités 19 à 25).

Sans ce compte, SQL Server démarre mais **aucun job ne s'exécute**, et les alertes ne fonctionnent pas.

1.3 Pourquoi deux comptes distincts ?

Principe de moindre privilège : chaque service a son propre compte, avec ses propres permissions. Si `SVC_SQL_Engine` est compromis, l'attaquant n'a **pas** accès aux droits supplémentaires de `SVC_SQL_Agent`, et inversement.

Isolation : si le service SQL Agent doit être redémarré (ce qui arrive), il peut l'être sans impacter le moteur SQL Server.

Traçabilité : les événements Windows et les logs SQL Server distinguent clairement les actions du moteur et celles de l'agent.

Recommandation Microsoft : depuis SQL Server 2012, Microsoft recommande d'utiliser des **comptes de service distincts** pour le moteur et l'agent.

---

2. Le mécanisme qu'on vient de mettre en place

2.1 Étape 1 — Création des comptes AD

Où : DC03.
Script : `3.2.0.create_sql_service_accounts.ps1`.
Ce qui a été fait :

- Création de deux comptes utilisateurs dans l'OU `ServiceAccounts`.
- `SVC_SQL_Engine` et `SVC_SQL_Agent`.
- `PasswordNeverExpires = true` (les comptes de service ne doivent **jamais** expirer).
- `Enabled = true` (comptes actifs).

Résultat : les deux comptes existent dans Active Directory.

2.2 Étape 2 — Attribution des privilèges locaux

Où : SQLS01 et SQLS02.
Script : `3.2.1.grant_service_logon_rights.ps1`.
Ce qui a été fait :

- Attribution du privilège `SeServiceLogonRight` (Log on as a service).
- Attribution du privilège `SeManageVolumePrivilege` (Perform volume maintenance tasks).

Résultat : les deux comptes ont les droits nécessaires sur **chaque nœud** SQL Server.

2.3 Pourquoi les deux privilèges sont indispensables

2.3.1 `SeServiceLogonRight` (Log on as a service)

Ce que c'est : le droit pour un compte de **démarrer un service Windows**.

Pourquoi c'est nécessaire :

- Sans ce droit, Windows **refuse** au compte de se connecter en tant que service.

- Le service `MSSQLSERVER` **ne peut pas démarrer**.
- L'installation SQL Server **échoue** avec une erreur du type « Le service n'a pas pu démarrer ».

Quand c'est utilisé :

- À chaque **démarrage de Windows** (le service SQL Server démarre automatiquement).
- À chaque **redémarrage manuel** du service.
- À chaque **failover** du cluster (le service redémarre sur l'autre nœud).

Sans ce privilège : SQL Server **ne peut pas démarrer**.

2.3.2 `SeManageVolumePrivilege` (Perform volume maintenance tasks)

Ce que c'est : le droit d'**accéder directement au disque**, sans passer par le cache du système de fichiers.

Pourquoi c'est nécessaire :

- SQL Server utilise ce privilège pour **l'instant file initialization** (IFI).
- Quand SQL Server crée ou agrandit un fichier de données, il doit **écrire des zéros** sur tout l'espace alloué (pour éviter que des données d'anciens fichiers ne soient lisibles).
- **Sans IFI**, cette opération prend du temps proportionnel à la taille du fichier.
- **Avec IFI**, SQL Server alloue l'espace **instantanément** et écrit les données au fur et à mesure.

Exemple concret :

- Création d'un fichier de 10 GB **sans IFI** : ~30 secondes à 2 minutes (écriture de zéros).
- Création d'un fichier de 10 GB **avec IFI** : ~instantané.

Quand c'est utilisé :

- À la **création** d'une base de données.
- À l'**agrandissement** d'un fichier de données (croissance automatique).
- À la **restauration** d'une base de données (création des fichiers).
- À la **création de fichiers TempDB** au démarrage de SQL Server.

Sans ce privilège : SQL Server fonctionne, mais **les opérations de création/agrandissement de fichiers sont lentes**.

Impact sur les performances : sur un serveur qui héberge 13 bases de données + ASPState, l'absence d'IFI peut transformer une restauration de 5 minutes en 1 heure.

2.4 Le mécanisme `secedit` — comment ça marche

`secedit` est l'outil Windows qui gère la **stratégie de sécurité locale**. Il stocke les privilèges dans la base `%windir%\security\database\secedit.sdb`.

Ce que fait le script `3.2.1` :

1. **Export** de la configuration actuelle :

text

```
secedit /export /cfg "$env:TEMP\secpol.cfg"
```

Cela produit un fichier `.cfg` qui contient les privilèges actuels, par exemple :

text

```
SeServiceLogonRight = *S-1-5-80-0
SeManageVolumePrivilege = *S-1-5-32-544
```

2. **Modification** du fichier `.cfg` pour ajouter les SID des comptes de service :
   - `SVC_SQL_Engine` → `*S-1-5-21-...-1229`
   - `SVC_SQL_Agent` → `*S-1-5-21-...-1230`
3. **Application** de la configuration modifiée :

   text

   ```
   secedit /configure /db secedit.sdb /cfg "$env:TEMP\secpol.cfg" /areas
   USER_RIGHTS
   ```

   Cela réapplique les privilèges avec les nouveaux SID.

2.5 Le message d'avertissement

Message :
text

```
Le mappage entre les noms de compte et les ID de sécurité n'a pas été
effectué.
La tâche s'est terminée avec une erreur.
```

Ce que cela signifie : `secedit` a rencontré un SID qu'il ne pouvait pas résoudre. Dans notre cas, c'est **`S-1-5-80-0`** qui correspond au groupe système **`NT SERVICE\ALL SERVICES`**. Ce SID est interne à Windows et n'est pas résolvable par `secedit` en mode batch.

Impact réel : **aucun**. Les SID des comptes de service ont bien été ajoutés, comme le prouve la vérification finale.

C'est un faux positif bénin. Windows applique quand même les privilèges pour les SID résolubles.

---

3. Récapitulatif du mécanisme

Élément	Rôle	Où
`SVC_SQL_Engine`	Identité du service SQL Server (moteur)	AD (DC03)
`SVC_SQL_Agent`	Identité du service SQL Server Agent	AD (DC03)
`SeServiceLogonRight`	Permet aux services de démarrer	SQLS01, SQLS02
`SeManageVolumePrivilege`	Active l'instant file initialization	SQLS01, SQLS02
`secedit`	Outil Windows pour gérer les privilèges	SQLS01, SQLS02

---

4. Quand ces privilèges sont utilisés

4.1 `SeServiceLogonRight`

Événement	Fréquence
Démarrage de Windows	À chaque boot
Redémarrage du service SQL Server	Ponctuel
Failover du cluster Always On	À chaque bascule
Mise à jour de SQL Server	Lors des patchs

Sans ce privilège : SQL Server ne peut pas démarrer du tout.

4.2 `SeManageVolumePrivilege`

Événement	Impact
Création d'une base de données	Accélération
Agrandissement d'un fichier	Accélération
Restauration d'une base	Accélération forte
Création des fichiers TempDB	Accélération au démarrage

Sans ce privilège : SQL Server fonctionne, mais les opérations de fichiers sont lentes.

---

5. Ce qu'il faut retenir

Les deux comptes de service et les deux privilèges forment le socle de sécurité et de performance de SQL Server.

- **`SVC_SQL_Engine`** : identité du moteur SQL Server.
- **`SVC_SQL_Agent`** : identité de l'agent SQL Server.
- **`SeServiceLogonRight`** : permet aux services de démarrer.
- **`SeManageVolumePrivilege`** : active l'instant file initialization.

Sans ces éléments, SQL Server ne peut pas démarrer ou fonctionne avec des performances dégradées.

C'est la raison pour laquelle ces étapes sont exécutées AVANT l'installation de SQL Server.

---

6. Prochaine étape

Maintenant que le socle est en place, on peut passer à l'installation de SQL Server (`3.3.0`).